

Sentry critical health monitoring — test flags

Feature reference · TScopier worker · 2026-08-31

STATUS	Implemented — staging-only	ENVIRONMENT	Staging (refused in production)
COMPONENT	worker/src — Sentry observability	SCOPE	FxSocket outage, worker heartbeat, critical exception
FILES	testFlags.ts , fxsocketWsClient.ts , workerHeartbeat.ts , index.ts	STATUS	Ready for staging validation

1. Executive summary (plain English)

The TScopier worker already sends alerts to Sentry when something critical breaks — for example when the FxSocket data connection stays down too long, when the worker process stops reporting, or when the process crashes. To make sure those alerts actually arrive, we added three small switches that intentionally force a failure on staging. You turn a switch on in Railway, watch Sentry for the expected alert, then turn the switch off and everything returns to normal. The switches do nothing on the live production system, so testing can never affect real customers.

2. What is monitored

The worker covers five critical failure domains. Four now have monitoring and test switches; one does not yet have monitoring.

DOMAIN	MONITORING	TEST SWITCH
FxSocket / API disconnected for a sustained period	Yes	FXSOCKET_TEST_FORCE_DISCONNECT
Worker process stops / heartbeat disappears	Yes	WORKER_HEARTBEAT_TEST_FORCE_STOP
Critical worker / runtime exception	Yes	CRITICAL_EXCEPTION_TEST_FORCE
System-wide trade execution failure	Yes	TRADE_PIPELINE_TEST_FORCE_FAILURE
Cron / background job stopped	No (planned)	—

3. The switches

All switches default to **off** (system fully operational, alerts active). They are **ignored in a production environment**, so a staging setting can never be copied into production by mistake. Turning a switch on forces the failure; turning it off (or removing it) restores normal behaviour immediately — no worker restart is needed for the FxSocket or heartbeat switches.

SWITCH	DEFAULT	WHAT IT DOES WHEN ENABLED	SENTRY ALERT EXPECTED
FXSOCKET_TEST_FORCE_DISCONNECT	false	Immediately closes the FxSocket WebSocket connection(s). Socket stays down until the flag is set back to false.	After the outage grace (default 60s): Critical Issue <code>FXSOCKET_SOCKET_SUSTAINED_OUTAGE</code> .
WORKER_HEARTBEAT_TEST_FORCE_STOP	false	Stops the worker from sending Sentry check-in pings each minute.	After the configured margin (default 2 min): Monitor Down / missed check-ins .
CRITICAL_EXCEPTION_TEST_FORCE	false	Throws an intentional uncaught exception 1 second after startup.	Error Issue <code>UNCAUGHT_EXCEPTION</code> ; heartbeat also stops → Monitor Down.
TRADE_PIPELINE_TEST_FORCE_FAILURE	false	Forces all trade executions to fail so the trade-pipeline monitor can fire.	Critical Issue <code>TRADE_PIPELINE_SYSTEMIC_FAILURE</code> once the failure rate exceeds the threshold over the window.

Trade-pipeline monitor (real monitoring, active in production): the monitor counts trade successes and failures over a rolling window and alerts when the failure rate reaches `TRADE_PIPELINE_FAILURE_THRESHOLD_PCT` (default 50%) after at least 10 executions. Only the forced-failure injection is a staging test flag.

Careful: `CRITICAL_EXCEPTION_TEST_FORCE` crashes the worker process (that is its purpose) and Railway restarts it. Remove the switch after the test, otherwise the worker will keep crash-looping.

4. How to test on staging

Set the switch in Railway → Variables, watch Sentry, then set the switch back to `false` (or remove it).

SCENARIO	SWITCH TO SET	EXPECTED RESULT
FxSocket sustained disconnect	<code>FXSOCKET_TEST_FORCE_DISCONNECT=true</code>	Socket stays down. Critical Issue <code>FXSOCKET_SOCKET_SUSTAINED_OUTAGE</code> fires after the grace period (60s).
Recovery	Set <code>FXSOCKET_TEST_FORCE_DISCONNECT=false</code>	Socket reconnects; recovery breadcrumb recorded; issue resolves.
Second outage after recovery	Add <code>SENTRY_CRITICAL_HEALTH_COOLDOWN_MS=0</code> , repeat the sustained-disconnect test	A new alert is allowed (not suppressed by the cooldown).
Heartbeat failure	<code>WORKER_HEARTBEAT_TEST_FORCE_STOP=true</code>	Monitor Down after the margin.
Critical exception	<code>CRITICAL_EXCEPTION_TEST_FORCE=true</code>	Error Issue <code>UNCAUGHT_EXCEPTION</code> + Monitor Down. Remove switch after.
Trade pipeline failure	<code>TRADE_PIPELINE_TEST_FORCE_FAILURE=true</code>	Critical Issue <code>TRADE_PIPELINE_SYSTEMIC_FAILURE</code> once enough executions fail at/above the threshold within the window.

5. Production safety

All switches are read through a guard (`worker/src/testFlags.ts`) that refuses to act in a production environment. Production is detected from `SENTRY_ENVIRONMENT` , `RAILWAY_ENVIRONMENT_NAME` , or `NODE_ENV` being `production / prod` ; when none is set the default is treated as production, so the switches stay off unless a staging environment name is explicitly set. This means a staging environment variable set can never crash-loop or disconnect the live trade worker.

6. Files changed

- `worker/src/testFlags.ts` (+ `testFlags.test.ts`) — production guard and flag helpers.
- `worker/src/fxsocketWsClient.ts` — force-disconnect path in `connect()` + scheduled reconnect.
- `worker/src/observability/workerHeartbeat.ts` — skip check-ins when the stop switch is on.
- `worker/src/index.ts` — forced uncaught exception path.

- `worker/.env.example` , `docs/sentry-critical-health-plan.md` — documentation.

7. Verification

- Worker build (typecheck): `npm --prefix worker run build` — passes.
- Targeted tests: 68 node:test cases pass (FxSocket health, test flags, worker heartbeat, critical health, Sentry).
- Deploy to staging, run the scenario checks above, and confirm each expected alert arrives in Sentry before promoting to production.

8. Deployment status

Implemented and reviewed (PASS_WITH_NOTES; all actionable findings resolved). Not yet deployed — pending staging validation. To deploy, set `SENTRY_ENABLED=true` and `SENTRY_DSN` on the staging worker service per the existing worker Sentry policy, then run the scenario checks.

9. Follow-ups

1. Run the full scenario matrix on staging and record the observed Sentry alerts for each switch (including the trade-pipeline failure).
2. Build monitoring for the remaining domain (cron/job stall) and add a matching test switch.
3. Reconsider the production-name detection (currently `production` / `prod`) if a future production environment uses a different name.